Day 5 – Making RAG Work Well (Quality, Ranking, and Guardrails)

Day 5 focuses on turning a basic RAG (Retrieval Augmented Generation) system into a reliable, production-ready system. The main idea is that most RAG failures happen before the LLM is called: during retrieval, ranking, filtering, and prompt design.

1. The Core Problem

In a naive RAG system, we retrieve top-K chunks from a vector database and send all of them to the LLM. However, vector similarity does not always mean true relevance. This often results in: - Noisy context - Missing the exact answer chunk - The LLM giving vague or incorrect answers

2. Re-ranking: The Second Brain

Instead of trusting vector search blindly, a better pipeline is: Retrieve (top 20 or 30) → Re-rank by true relevance → Keep only top 3–5 → Send to LLM. The re-ranker (or an LLM used as a judge) scores each chunk against the user question and filters out weak or irrelevant ones. This dramatically improves answer quality.

Example: HR Policy Bot

Question: "How many casual leaves do I get per year?" Vector search may return: - "Employees are entitled to 12 casual leaves per year." (Correct) - "There are 12 public holidays in 2025." (Wrong but similar number) - "Employees may take leave with manager approval." (Semi-related) Re-ranking keeps only the truly relevant chunk about casual leaves, so the LLM answers correctly and confidently.

3. Grounding the LLM with a Proper Prompt

A strong RAG prompt must tell the LLM: - Use ONLY the provided context to answer. - If the answer is not in the context, say "I don't know based on the provided documents." This prevents hallucinations and turns the LLM into a reader and summarizer instead of a guesser.

4. Context Window Budget (How Much to Send)

LLMs have limited context windows and larger prompts cost more and can reduce accuracy. Sending too many chunks: - Adds noise - Confuses the model - Increases cost and latency Best practice: - Send only the top 3–5 high-quality chunks after re-ranking - Each chunk should be reasonably sized (for example, 300–700 tokens)

Example: Codebase Assistant

Question: "Where is JWT validation implemented?" Naive system: - Sends 15 chunks from auth.py, middleware.py, settings.py, utils.py, etc. - LLM gives a long, vague answer mentioning many files. Improved system: - Retrieves 20 chunks - Re-ranks and keeps only chunks from auth.py and middleware.py that actually validate JWT - Sends only those to the LLM - LLM answers precisely where and how JWT validation is implemented.

5. Common Failure Modes and Fixes

Common problems: - Bad chunking: chunks are too big or too small → fix by better chunk size and overlap - Noisy retrieval: irrelevant chunks included → fix with re-ranking and filtering - Hallucinations: LLM guesses when answer is missing → fix with grounding prompt - Too many chunks: slow and expensive → fix by shrinking context to top few chunks

6. Practical Debugging Tip

Always log and inspect: - The retrieved chunks from the vector database - The final chunks sent to the LLM - The final prompt In most RAG systems, the bug becomes obvious just by reading these three things.

7. One-Line Truth of Day 5

The quality of a RAG system is mostly decided before the LLM is called: by retrieval, re-ranking, filtering, and instructions.